

同濟大學

TONGJI UNIVERSITY

毕业设计（论文）

课题名称 基于 AI Agent 的建筑幕墙韧性评估软件系统设计与实现

副标题

学院 计算机科学与技术学院

专业 软件工程

学生姓名 林继申

学号 2250758

指导教师 黄杰

日期 2026 年 5 月 1 日

同济大学本科毕业设计（论文）信息说明页

课题名称：基于 AI Agent 的建筑幕墙韧性评估软件系统设计与实现

成果类型：☒ 毕业设计 ☐ 毕业论文

学科专业：软件工程

内容简述（请用 300 字以内简要概述）：

本毕业设计面向建筑幕墙韧性评估中人工巡检效率低、结果主观、图像数据难以规模化处理和报告难以系统生成的问题，设计并实现一套基于 AI Agent 的软件系统。系统以项目级图像资产管理为基础，由 Agent 自主判断每张图像需要执行的原子检测能力，并通过分层信息整合策略先生成单图总结，再融合为项目级评估报告。工程上采用前后端分离、gRPC、RocketMQ、WebSocket、MySQL、Redis 与 MinIO 等技术，形成可部署、可追踪、可扩展的端到端原型。本文重点在于 Agent 赋能工程链路，而非单个视觉模型本身。

毕业设计：主要图纸 5 张，正文总字数 27730 字

随附资料：

1. 建筑幕墙韧性评估软件系统源代码；
2. 数据库定义、接口定义与部署配置文件；
3. 图像检测样例、运行说明与系统演示材料。

(请列出所有提交的支撑材料名称，如：毕业设计-全套图纸、毕业作品、计算书、程序代码、附录等)

基于 AI Agent 的建筑幕墙韧性评估软件系统设计与实现

摘 要

建筑幕墙是现代建筑的重要外围护结构，其服役状态直接影响建筑安全、耐久性和运维质量。现实工程中，幕墙巡检长期依赖人工经验，存在效率低、主观性强、难以处理海量图像和难以形成系统化报告等问题。已有计算机视觉方法能够完成裂缝、锈蚀、破损等局部缺陷检测，但多数方案停留在单模型、单图像、单缺陷识别层面，难以解决“从项目级图像资产到建筑级韧性报告”的完整业务链路问题。本文面向这一现实痛点，设计并实现一套基于 AI Agent 的建筑幕墙韧性评估软件系统。

本文工作的重点不在于重新提出某个检测模型，而在于构建一个能够把既有视觉检测能力组织起来、让 Agent 参与任务规划与信息整合的工程框架。系统首先管理项目基础信息和幕墙图像资产，然后由分类 Agent 根据图像内容自主判断需要执行的原子检测能力，包括金属锈蚀、石材裂缝、石材污渍、玻璃平整度和玻璃爆裂。检测完成后，系统进一步提出分层信息整合策略：先将每张图像的多源检测结果生成单图总结，再将所有单图总结、项目背景、图像分组和人工复核意见融合为项目级韧性评估报告。该策略将大语言模型从简单文本生成工具扩展为“路由、压缩、汇总”的智能节点，缓解了海量检测数据直接输入模型带来的上下文过载和重点淹没问题。

在工程实现上，系统采用 React 前端、Golang 后端、gRPC 服务通信、RocketMQ 异步任务触发、WebSocket 实时进度推送、MySQL 结构化存储、Redis 缓存和 MinIO 对象存储。工程链路为 Agent 能力提供稳定运行环境，使分类、检测、总结和报告生成过程可追踪、可回调、可恢复和可展示。最终系统形成从图像上传、智能检测、人工复核到报告生成的完整闭环，为建筑幕墙运维提供了一种兼具工程可落地性和智能决策能力的解决方案。

关键词：AI Agent，建筑幕墙，韧性评估，任务编排

Design and Implementation of an AI Agent-Based Software System for Building Curtain Wall Resilience Assessment

ABSTRACT

Building curtain walls are important envelope structures in modern buildings, and their service condition directly affects building safety, durability, and operation quality. In real engineering practice, curtain wall inspection has long relied on manual experience, leading to low efficiency, strong subjectivity, difficulty in processing massive image data, and difficulty in producing systematic reports. Existing computer vision methods can detect local defects such as cracks, corrosion, and breakage, but most solutions remain at the level of single-model, single-image, and single-defect recognition. They are still insufficient for solving the complete business workflow from project-level image assets to building-level resilience assessment reports. To address this practical pain point, this thesis designs and implements an AI Agent-based software system for building curtain wall resilience assessment.

The focus of this work is not to propose a new detection model, but to build an engineering framework that organizes existing visual detection capabilities and enables Agents to participate in task planning and information integration. The system first manages project profile information and curtain wall image assets. Then, a classification Agent autonomously determines which atomic detection capabilities should be executed according to image content, including metal corrosion detection, stone crack detection, stone stain detection, glass flatness detection, and glass spalling detection. After detection is completed, the system further proposes a hierarchical information integration strategy: multi-source detection results of each image are first summarized into an image-level summary, and then all image-level summaries, project background, image groups, and human review comments are integrated into a project-level resilience assessment report. This strategy extends large language models from simple text generation tools

into intelligent nodes for routing, compression, and aggregation, alleviating the context overload and key-information dilution caused by directly feeding massive detection data into the model.

In terms of engineering implementation, the system adopts a React frontend, Golang backend services, gRPC service communication, RocketMQ-based asynchronous task triggering, WebSocket-based real-time progress pushing, MySQL structured storage, Redis caching, and MinIO object storage. The engineering workflow provides a stable runtime environment for Agent capabilities, making classification, detection, summarization, and report generation traceable, callback-driven, recoverable, and displayable. The final system forms a complete closed loop from image upload, intelligent detection, and human review to report generation, providing a solution for building curtain wall operation and maintenance that combines engineering feasibility with intelligent decision-making capability.

Key words: AI Agent, Building Curtain Wall, Resilience Assessment, Task Orchestration

目 录

1	绪论	1
1.1	研究背景	1
1.2	现实痛点与研究问题	1
1.3	为什么需要 Agent	2
1.4	本文技术定位	3
1.5	本文研究目标	4
1.6	研究内容与贡献	4
1.7	论文组织结构	5
2	从幕墙检测到 Agent 工程化的研究基础	6
2.1	建筑幕墙巡检与数字化运维	6
2.2	幕墙缺陷检测技术	7
2.3	从单点算法到项目级评估的差距	7
2.4	AI Agent 的发展与工程范式	8
2.5	Agent 工程化中的可控性问题	9
2.6	工程软件中的异步任务编排	9
2.7	报告生成与信息整合相关工作	10
2.8	现有研究不足与本文切入点	11
3	问题建模与系统需求分析	12
3.1	业务对象建模	12
3.2	用户流程建模	12
3.3	业务闭环中的角色与责任	13
3.4	典型用例分析	14
3.5	功能需求分析	14
3.5.1	项目与权限需求	14
3.5.2	图像资产需求	14
3.5.3	智能检测需求	15
3.5.4	复核与报告需求	15
3.6	阶段化需求分析	15
3.7	非功能需求分析	16
3.7.1	可扩展性	16
3.7.2	一致性	16
3.7.3	可观测性	16
3.7.4	可部署性	16
3.7.5	用户体验	17
3.8	关键挑战	17
3.9	数据流分析	18
3.10	约束条件	18
3.11	需求与设计目标映射	18
4	面向幕墙评估的 Agent 自主调度与分层信息整合方法	20

4.1	方法概述	20
4.2	Agent 在本文中的角色定义	20
4.3	受控 Agent 调度机制	21
4.4	为什么不是端到端大模型	22
4.5	原子检测能力工具化	22
4.6	检测任务状态流转	23
4.7	分层信息整合策略	24
4.8	单图总结设计	25
4.9	项目报告生成设计	25
4.10	人机协同机制	25
4.11	方法优势分析	26
4.12	方法流程小结	26
5	支撑 Agent 落地的系统架构与工程实现	28
5.1	总体架构设计	28
5.2	通信与协议设计	28
5.3	共享协议与枚举治理	29
5.4	数据库与状态机实现	30
5.5	Detection Worker 实现	31
5.6	对象存储与产物管理	32
5.7	Reasoning 服务实现	33
5.8	前端可视化实现	33
5.9	部署实现	34
5.10	工程复杂度总结	35
6	系统验证与应用分析	36
6.1	验证目标	36
6.2	功能验证	36
6.3	Agent 调度验证	37
6.4	分层信息整合验证	37
6.5	状态一致性验证	38
6.6	部署验证	38
6.7	应用价值分析	39
6.8	局限性分析	39
6.9	本章小结	40
7	总结与展望	41
7.1	全文总结	41
7.2	主要成果	41
7.3	不足之处	42
7.4	未来工作	42
	参考文献	43
	附录	45
	A 系统模块与接口补充说明	45
	A.1 服务模块	45

A.2 原子检测任务	45
谢辞	46

装
订
线

1 绪论

1.1 研究背景

建筑幕墙是现代建筑的重要外围护结构，承担围护、采光、隔热、防水、防风压、节能和外观表达等多重功能。随着城市中大量公共建筑、商业建筑和高层建筑进入长期服役阶段，幕墙系统的安全性、耐久性和可维护性逐渐成为建筑运维中的重要问题。幕墙材料长期暴露在风、雨、温差、紫外线、污染物和偶发冲击环境中，可能出现玻璃爆裂、玻璃平整度异常、金属锈蚀、石材裂缝、石材污渍以及连接构件退化等问题。这些问题如果不能及时发现和评估，不仅会影响建筑外观，还可能对围护性能、使用安全和维护成本产生持续影响。

在建筑运维数字化和 BIM 技术发展的背景下，幕墙评估的目标正在从“发现某个缺陷”转向“理解建筑外围护系统的整体状态”。韧性概念强调系统在扰动或退化条件下维持关键功能、恢复功能和适应环境变化的能力。对于建筑幕墙而言，韧性评估不仅需要知道某张图像中是否存在缺陷，还需要回答缺陷分布在哪些区域、不同材料上的损伤类型是否具有聚集趋势、局部损伤对整体运维决策有何意义、后续维护应优先关注哪些问题等更高层次的问题。

现实工程中，幕墙巡检仍然大量依赖人工经验。检测人员需要面对高空作业、大面积立面、海量图像、复杂材料和不一致的现场环境。即便无人机和高清相机降低了图像采集难度，后续的图像整理、材质判断、缺陷识别、结果汇总和报告撰写仍然消耗大量人工。实际业务中经常出现“图像采集已经完成，但评估报告迟迟难以形成”的情况，其根源在于图像数据和评估结论之间缺少自动化、结构化、可追踪的信息处理链路。

1.2 现实痛点与研究问题

本文关注的现实痛点可以概括为三个层次。第一个层次是数据规模问题。一个建筑项目可能包含大量幕墙图像，图像之间还需要按照立面、楼层、区域或采集批次进行组织。若缺少图像资产管理能力，图像只能作为普通文件存在，难以支撑后续检索、复核、状态追踪和报告生成。

第二个层次是任务选择问题。幕墙图像中可能包含玻璃、金属、石材或混凝土等不同材料，不同材料对应的检测能力不同。传统固定流程会对每张图像执行全部检测模型，这会造成计算浪费和语义噪声。例如，对玻璃幕墙图像执行石材裂缝检测并无实际意义，对石材图像执行玻璃平整度检测也难以产生可信结论。人工选择检测能力虽然可行，但在批量场景下效率较低，且缺少一致性。

第三个层次是报告生成问题。检测模型通常输出区域坐标、像素占比、置信度、掩码图像和浮层图像等底层结果。这些结果对算法复核有价值，但并不能直接构成建筑运维人员可读的韧性评估报告。如果直接把所有检测结果拼接给大语言模型，海量数据内容会造成上下文过长、信息重复、重点不清和报告生成不稳定。因此，系统需要一种从原子检测结果到图像级总结，再到项目级报告的中间组织机制。

上述三个层次共同构成本文的研究问题：如何设计一套面向建筑幕墙韧性评估的软件系统，使其能够管理项目级图像资产，基于图像内容自主选择检测能力，调度原子检测工具，并将多源检测结果逐层整合为项目级评估报告。

1.3 为什么需要 Agent

传统软件系统擅长执行确定流程，但面对“先判断场景，再选择工具，再综合输出结论”的任务时，往往需要大量规则和人工配置。建筑幕墙检测正属于这种情况。图像是否属于幕墙、主要材料是什么、应该运行哪些检测能力、检测结果如何转化为工程语言，这些问题具有一定语义判断和上下文理解需求。单纯依靠硬编码规则难以覆盖复杂场景，单纯依靠检测模型又无法完成任务规划和报告表达。

AI Agent 范式为该问题提供了新的解决思路。近年来，学术界和工业界逐渐将大语言模型从“文本生成器”扩展为“任务规划和工具调用中心”。ReAct 方法强调推理和行动交替进行，使模型能够在推理过程中调用工具并利用工具反馈更新计划^[1]；Toolformer 讨论语言模型学习使用外部 API 的能力^[2]；AutoGen 等框架进一步展示了多 Agent 协作和可编程智能工作流的工程价值^[3]。这些研究表明，Agent 的核心价值不只是输出自然语言，而是把理解、规划、工具调用和结果整合连接起来。

本文将 Agent 引入建筑幕墙韧性评估，并不是为了给系统增加一个聊天入口，而是为了让 Agent 承担三个具体职责：在检测前完成任务路由，在检测后完成图像级语义压

缩，在项目结束时完成报告级信息融合。这样的设计使 Agent 真正嵌入业务链路，而不是作为孤立的问答功能存在。

如果采用传统规则系统，也可以通过人工配置材料和检测任务之间的映射关系。例如，当用户选择“玻璃幕墙”时，系统固定执行玻璃平整度和玻璃爆裂检测；当用户选择“石材幕墙”时，系统固定执行裂缝和污渍检测。但是现实图像往往并不如此简单。一张图像可能同时包含玻璃和金属框架，也可能包含非幕墙背景、反射、遮挡或局部构件。完全依赖用户手动选择材料，会降低批量处理效率；完全依赖硬编码规则，又难以处理复杂视觉语义。Agent 的价值正在于它可以在有限工具集合内完成语义判断，从而把复杂场景判断转化为工程系统可执行的任务代码。

如果直接让大语言模型生成最终报告，也存在明显问题。幕墙项目可能包含多张图像，每张图像又包含多个检测任务和多个区域结果。原始结果中大量坐标、像素、置信度和图像产物信息会干扰报告生成。本文没有让 Agent 一步完成所有任务，而是将其拆分为三个受控节点：分类路由、单图总结和项目报告。这种设计既符合当前 Agent 工程化中的工具调用思想，也符合建筑评估从局部观察到整体判断的认知过程。

1.4 本文技术定位

本文是一项以工程系统为载体、以 Agent 能力为核心亮点的毕业设计。工程系统本身采用的是相对成熟的技术组合，例如前后端分离、gRPC、消息队列、对象存储和数据库事务。这些技术的价值不在于新颖，而在于为 Agent 能力提供稳定运行环境。没有工程链路，Agent 只能处理单次输入；有了工程链路，Agent 的判断才能驱动任务创建、状态推进、结果落库和报告生成。

因此，本文的技术定位不是“实现一个检测模型”，也不是“实现一个通用后台管理系统”，而是“构建一个 Agent 驱动的幕墙韧性评估流程”。检测模型是工具，工程架构是载体，Agent 调度和分层信息整合是核心方法。这样的定位能够更准确地反映课题工作量和技術特色。

1.5 本文研究目标

本文的研究目标是设计并实现一套基于 AI Agent 的建筑幕墙韧性评估软件系统。系统需要完成从项目创建、图像资产管理、智能检测、人工复核到报告生成的完整流程。在智能检测阶段，系统需要通过分类 Agent 判断每张图像需要执行哪些原子检测能力；在原子检测阶段，系统需要稳定调度金属锈蚀、石材裂缝、石材污渍、玻璃平整度和玻璃爆裂五类检测能力；在信息整合阶段，系统需要将单张图像的检测结果生成图像级总结，再将所有图像总结和复核信息汇总为项目级报告。

从工程目标看，系统需要具备清晰模块边界和可部署性。前端需要提供完整项目流程和结果展示，后端需要支持长耗时任务、异步回调、实时状态推送、数据库事务和对象存储。工程实现不是本文的唯一亮点，但它是 Agent 能力落地的必要保障。没有稳定工程链路，Agent 调度和报告生成只能停留在脚本演示阶段，无法成为实际可用的软件系统。

1.6 研究内容与贡献

本文主要研究内容包括：第一，调研建筑幕墙检测与 AI Agent 应用范式，分析现有单点检测方案在项目级评估中的不足；第二，建立幕墙韧性评估软件的问题模型，明确图像资产、检测任务、人工复核和报告生成之间的关系；第三，提出面向幕墙图像的 Agent 自主调度机制，使系统能够根据图像内容动态选择检测能力；第四，提出分层信息整合策略，将原子检测结果先转化为图像级总结，再融合为项目级报告；第五，完成可运行软件原型，实现前端交互、服务通信、异步任务、状态推送、对象存储和部署验证。

本文贡献主要体现在以下三个方面。首先，本文将 Agent 工具调用思想引入建筑幕墙评估场景，设计了受控输出的任务路由机制，使 Agent 的语义判断能够被工程状态机消费。其次，本文提出分层信息整合策略，用图像级总结作为项目级报告生成的中间语义层，解决海量检测结果直接汇总的上下文压力和信息噪声问题。最后，本文构建了完整工程闭环，将 Agent 调度、视觉检测、结构化落库、人工复核和报告生成连接为可部署系统，体现了工程实践与智能方法结合的价值。

1.7 论文组织结构

全文共七章。第一章介绍研究背景、现实痛点、Agent 引入动机、技术定位和本文贡献。第二章梳理建筑幕墙检测、计算机视觉、AI Agent 和报告生成相关工作，说明本文选题的技术背景。第三章对问题进行建模，分析系统需求、角色、数据对象和关键挑战。第四章详细阐述本文提出的 Agent 自主调度机制和分层信息整合策略。第五章介绍系统架构与工程实现，重点说明复杂工程链路如何支撑 Agent 能力落地。第六章进行系统验证与应用分析，说明系统在功能、流程和部署层面的完成情况。第七章总结全文工作并展望后续研究方向。

2 从幕墙检测到 Agent 工程化的研究基础

2.1 建筑幕墙巡检与数字化运维

建筑幕墙巡检长期以来依赖人工现场勘查。人工巡检能够结合经验进行综合判断，但在高层建筑、大面积幕墙和复杂环境下效率较低，且存在安全风险。随着无人机技术和图像采集设备的发展，基于无人机的建筑立面巡检逐渐成为重要方向。Freimuth 和 König 对无人机建筑检查的规划和执行流程进行了系统研究，说明无人机能够降低高空检测成本并提升数据采集效率^[4]。在此基础上，红外热成像、图像配准和 BIM 结合等方法进一步扩展了建筑立面检测的数据来源和空间定位能力^[5-6]。

但是，数字化采集并不等于智能化评估。无人机可以快速采集图像，但图像采集之后仍需要图像管理、缺陷检测、结果解释和报告生成。若缺少后续软件系统，采集到的图像仍然是离散资料，难以转化为运维决策。因此，建筑幕墙评估系统需要将采集后的图像组织为项目资产，并建立从图像到评估报告的完整处理流程。

从工程应用角度看，幕墙巡检的难点并不只在“看清图像”。采集设备能够提高图像获取效率，检测模型能够提高局部缺陷识别效率，但真正面向业主、运维单位和工程师的交付物通常是带有结论、依据和建议的评估报告。这意味着系统需要同时处理视觉证据、结构化指标、项目语义和文本表达。若系统只停留在缺陷框或掩码图层面，仍然需要人工把这些结果重新解释、筛选、汇总和撰写。因此，图像检测之后的组织、解释和报告生成，是幕墙数字化运维走向智能化运维的关键环节。

此外，幕墙工程对象具有明显的项目属性。同一种缺陷出现在不同区域，其工程意义可能不同；同一张图像中的不同材料，也可能对应不同检测任务。传统视觉检测研究往往把图像视作独立样本，而实际运维更关心项目内图像之间的关系、图像组与建筑区域的关系、缺陷分布与维护优先级之间的关系。本文的系统设计正是围绕这种项目级应用需求展开。

2.2 幕墙缺陷检测技术

深度学习推动了图像缺陷检测的发展。卷积神经网络能够自动学习图像特征，目标检测和分割模型能够输出缺陷区域、类别和置信度^[7-8]。在建筑结构和幕墙场景中，裂缝检测、玻璃检测和表面缺陷识别等任务均已有较多研究。Ye 等人利用深度学习方法进行结构裂缝检测，为石材或混凝土裂缝识别提供了参考^[9]；Mei 等人研究真实场景中的玻璃检测问题，对玻璃幕墙区域分析具有启发意义^[10]。

从本文系统实现角度看，五类原子检测能力可以分别由 YOLO、传统计算机视觉算法或深度学习分类模型实现。金属锈蚀检测可以通过目标检测和分割获得锈蚀区域；石材裂缝检测可以通过分割方法获得裂缝掩码；石材污渍检测可以通过区域检测和局部分析得到污渍占比；玻璃平整度检测可以结合玻璃区域识别、边缘、直线、梯度和频域特征判断是否不平整；玻璃爆裂检测可以通过分类模型判断是否存在破损风险。

需要强调的是，本文并不把上述检测模型作为主要研究贡献。原因在于实际工程中，检测模型可以持续替换和迭代，系统真正需要解决的是“如何让这些检测能力被合理调用并形成评估报告”。因此，本文将检测模型抽象为原子能力，重点研究其工程封装、调度和结果整合。

这种定位并不削弱模型的重要性，而是重新定义模型在系统中的角色。对于工程软件而言，模型更像是一个可替换工具：当模型精度提高时，系统应能低成本接入；当新增缺陷类型时，系统应能扩展任务代码、数据表和展示逻辑；当某个模型执行失败时，系统应能隔离失败并保留其他任务结果。也就是说，模型能力需要被纳入工程治理，而不是以脚本形式孤立运行。本文后续提出的原子检测能力工具化，就是为了解决这一问题。

2.3 从单点算法到项目级评估的差距

已有幕墙缺陷检测研究通常以单一任务为目标，例如裂缝识别、玻璃检测或表面污渍检测。这类研究能够回答算法层面的问题：给定输入图像，模型是否能够准确输出目标区域。然而，项目级评估需要回答更复杂的问题：哪些图像需要执行哪些检测，多个检测结果之间如何互相补充，哪些缺陷对项目整体风险影响更大，报告中应如何表

达证据与结论之间的关系。

从单点算法到项目级评估之间至少存在四个缺口。第一是数据组织缺口，图像需要按照项目、图像组、图像和检测任务多层组织。第二是任务编排缺口，系统需要根据图像内容动态选择检测能力，而不是机械执行全部模型。第三是结果解释缺口，模型输出的像素和坐标指标需要转化为工程语言。第四是报告生成缺口，系统需要把多张图像的局部结论融合为项目级评估意见。本文的研究价值正是在这些缺口处构建桥梁。

如果只关注检测模型，系统会缺少业务闭环；如果只关注 Web 管理界面，系统又缺少智能决策能力。本文选择的路线是以工程系统组织检测模型，以 Agent 承担语义决策和信息整合，从而把二者连接为完整应用。

2.4 AI Agent 的发展与工程范式

大语言模型的兴起使 Agent 成为当前人工智能应用的重要范式。早期大语言模型主要用于文本生成和问答，而 Agent 更强调模型与外部工具、环境和用户目标之间的交互。ReAct 提出将推理和行动交替结合，使模型能够生成思考过程并执行任务动作^[1]。Toolformer 进一步说明语言模型可以学习何时调用工具、如何传递参数以及如何利用工具返回结果^[2]。这些工作共同说明，模型本身的语言能力需要通过工具调用机制转化为可执行能力。

工业界和开源社区也逐渐形成 Agent workflow 开发范式。AutoGen 通过多智能体对话组织复杂任务，说明 Agent 可以被看作具有不同角色和能力的可编程组件^[3]。相关综述指出，大语言模型 Agent 可以应用于单智能体、多智能体和人机协作等场景，其关键能力包括规划、记忆、工具使用和反馈学习^[11]。在工程实践中，Agent 往往不会完全自由行动，而是被放置在受控 workflow 中，通过结构化输入输出、工具白名单和状态管理保证系统可控。

本文正是基于这一认识，将 Agent 设计为业务链路中的受控智能节点。分类 Agent 不是自由选择任意工具，而是在预定义五类任务代码中选择；总结 Agent 不是直接修改系统状态，而是通过回调接口提交总结结果；项目报告 Agent 不是自行读取数据库，而是接收由 BIZ 服务组织好的项目上下文。这样的设计符合当前 Agent 工程化趋势：让模型承担语义理解和决策任务，让工程系统负责边界和状态。

表 2.1 大语言模型应用范式的演进

范式	主要特征	局限与本文启发
文本生成	模型根据提示词直接输出自然语言。	适合问答和撰写,但难以与业务状态和外部工具形成闭环。
检索增强	先检索外部知识,再生成回答。	能补充上下文,但仍偏向知识问答,不直接解决任务调度问题。
工具调用	模型根据目标选择外部 API 或工具。	适合连接模型与工程能力,但需要严格约束工具集合和参数。
工作流 Agent	Agent 被放入确定业务流程,承担局部决策。	更适合工程落地,本文采用该路线实现检测路由和信息整合。

表 2.1 展示了大语言模型应用范式从文本生成到工作流 Agent 的变化。本文并不把 Agent 理解为一个完全自主的软件机器人,而是将其作为工程 workflow 中的智能决策节点。这样的设计更符合垂直行业系统的要求: 业务流程必须可解释, 数据状态必须可追踪, 外部工具调用必须可控, 失败情况必须可恢复。

2.5 Agent 工程化中的可控性问题

Agent 系统虽然具备更强的自主性,但也带来工程可控性问题。大语言模型输出具有概率性,可能出现格式不稳定、内容越界、幻觉或工具选择错误等情况。对于普通问答系统,这些问题可能只是回答质量问题;对于工程业务系统,如果 Agent 输出直接驱动数据库状态或任务调度,则可能造成错误流程。因此,Agent 工程化必须关注结构化输出、工具白名单、状态隔离和人工复核。

本文采用的思路是将 Agent 决策限制在有限动作空间内。分类 Agent 只能输出五种任务代码的组合;图像总结 Agent 的结果只作为总结字段保存,不直接修改检测状态;项目报告 Agent 接收的输入由 BIZ 服务生成,输出也通过固定回调进入数据库。这样做牺牲了一部分开放性,但换来了工程系统所需的确定性和可追踪性。对于建筑幕墙评估这种专业场景,受控 Agent 比完全开放 Agent 更适合落地。

2.6 工程软件中的异步任务编排

长耗时任务编排是智能检测系统必须解决的问题。模型推理、外部 Agent 调用、对象上传和报告生成都可能持续数秒到数分钟,若采用同步 HTTP 请求等待全部结果,系统容易超时,用户体验也较差。成熟工程系统通常采用任务队列、Worker、回调和状态

查询机制，将“启动任务”和“获得结果”解耦。

本文系统采用类似思想，但结合 Agent 场景进行了扩展。用户启动检测后，BIZ 服务只负责创建任务并入队；分类、推理和总结能力由不同 Activity 服务异步完成；完成后通过回调上报结果；前端通过 WebSocket 感知状态变化。该模式使 Agent 和模型调用可以作为后台任务运行，也使系统能够在失败时记录状态并支持重试。

异步任务编排对本文尤为重要，因为 Agent 调用和模型推理都具有不可忽略的不确定性。Agent 平台可能因为网络、限流或输出异常导致失败；视觉模型可能因为图像格式、模型加载或运行环境导致失败；对象存储上传也可能受到网络影响。如果所有步骤都放在一次同步请求中，任何环节失败都会导致整个请求失败，且用户无法获得过程反馈。通过任务状态机和回调机制，系统能够把这些不确定因素转化为可记录、可展示和可恢复的状态变化。

从软件工程角度看，异步任务不仅是性能优化，也是复杂业务建模方式。检测主任务、子检测任务、总结任务和报告任务之间存在依赖关系。只有把这些关系显式建模为数据库状态和 Worker 编排逻辑，系统才能保证“分类失败不继续检测”“子任务全部成功才总结”“无检测任务时直接成功”等业务规则被稳定执行。

2.7 报告生成与信息整合相关工作

大语言模型擅长自然语言生成，但在知识密集型任务中，输入信息的组织方式直接影响输出质量。检索增强生成通过外部知识库向模型提供上下文，缓解模型仅依赖参数知识导致的不可靠问题^[12]。与此类似，工程报告生成也需要将外部数据组织成适合模型处理的上下文。如果输入过少，报告缺少依据；如果输入过多，模型容易遗漏重点或产生不稳定输出。

建筑幕墙韧性评估报告属于典型的多源信息整合任务。其输入包括项目基础信息、图像组信息、原子检测指标、图像产物、区域级数据、用户复核评论和任务状态。直接把这些信息一次性提交给大语言模型并不合理，因为底层指标数量可能随图像数量和区域数量快速增长。本文提出分层信息整合策略，将报告生成拆分为图像级和项目级两个阶段。该策略并非简单压缩文本，而是根据工程评估逻辑先形成单图语义，再进行项目级融合。

2.8 现有研究不足与本文切入点

综合来看，现有研究在三个方面尚存在不足。第一，幕墙检测研究多关注单类缺陷识别，对图像资产管理、任务调度和报告生成关注不足。第二，Agent 研究多集中在通用工具调用、代码执行和问答任务，在建筑工程垂直场景中的系统化落地仍有探索空间。第三，大语言模型报告生成往往关注生成效果本身，而对输入信息如何分层组织、如何与数据库状态和工程流程结合讨论不足。

本文的切入点正是在上述空白之间建立连接：以幕墙检测为现实场景，以 Agent 作为任务调度和信息整合能力，以工程系统作为稳定运行载体，构建从图像资产到韧性报告的完整链路。这样既避免了单纯算法研究难以落地的问题，也避免了单纯工程系统缺少智能亮点的问题。

更具体地说，本文并不试图在单个模型指标上与已有算法竞争，而是研究如何在真实业务流程中组织这些算法。已有检测模型回答“这张图是否存在某类缺陷”，本文系统进一步回答“这张图应该检测什么”“这些检测结果如何变成图像总结”“所有图像如何形成项目报告”。这三个问题正是 Agent 能力与工程链路结合的空间。

因此，本文的研究切入点具有复合性。一方面，它需要具备工程系统的完整性，能够支持登录、项目管理、图像上传、状态推送、结果展示和部署运行；另一方面，它需要体现 Agent 方法的必要性，即在任务路由和报告生成中解决传统规则系统难以处理的语义判断问题。本文后续章节将围绕这一主线展开：先定义问题和需求，再提出 Agent 调度与分层整合方法，最后说明工程架构如何支撑该方法落地。

3 问题建模与系统需求分析

3.1 业务对象建模

为了将幕墙韧性评估从人工流程转化为软件系统，首先需要明确系统中的核心业务对象。本文将业务对象划分为项目、图像组、图像、检测任务、子检测任务、图像总结、人工复核和项目报告八类。项目是评估的最高组织单位，保存建筑名称、位置、建设年份、已知问题和评估目标等背景信息。图像组用于组织幕墙图像，既可以对应建筑立面，也可以对应楼层、采集区域或用户自定义分组。图像对象保存原图、尺寸、文件大小、元数据、上传状态和所属图像组。

检测任务以单张图像为单位。每张图像最多对应一个检测主任务，主任务记录整张图像的检测状态，并关联分类路由结果、五类子检测任务和图像总结任务。子检测任务对应具体原子能力，例如金属锈蚀或石材裂缝。图像总结是检测结果的语义压缩层，人工复核是用户对结果的补充判断，项目报告则是最终面向运维决策的输出。

这种对象划分使系统具备清晰的数据层次。项目提供背景，图像组提供空间或业务语义，图像提供分析对象，检测任务提供状态，子任务提供结构化事实，图像总结提供局部解释，人工复核提供人工补充，项目报告提供综合结论。本文后续的系统设计均围绕这一对象模型展开。

在该对象模型中，图像不是孤立输入，而是项目资产的一部分。检测任务也不是一次函数调用，而是围绕图像产生的业务记录。这样建模可以支持两个重要能力：第一，用户可以在任何时间回到项目查看历史检测结果；第二，系统可以在报告阶段追溯每个结论来自哪张图像、哪个检测任务和哪个区域。对于工程评估软件而言，可追溯性与检测能力同样重要。

3.2 用户流程建模

系统用户流程可以抽象为五个阶段。第一阶段是项目基础信息维护，用户录入建筑基本情况和评估目标。第二阶段是图像资产构建，用户上传图像并按组整理。第三阶段是 Agent 智能检测，系统自动创建检测任务、调用 Agent 和原子检测能力，并实时展

示进度。第四阶段是人工复核，用户查看检测结果、补充评论并标记结果准确性。第五阶段是报告生成，系统汇总前面阶段的信息并生成项目级评估报告。

各阶段之间存在严格的业务依赖。没有项目基础信息，报告缺少背景；没有图像资产，检测无法启动；检测未完成，复核无从进行；复核结束后，报告生成才能获得完整上下文。系统因此需要项目进度控制，避免用户在数据不完整时进入后续阶段。同时，为了降低用户等待成本，检测和报告生成等长耗时任务必须异步执行。

3.3 业务闭环中的角色与责任

系统中的主要角色可以分为使用者、业务服务、Agent 服务和检测模型服务。使用者负责创建项目、上传图像、查看检测结果、进行人工复核并确认报告。业务服务负责鉴权、数据持久化、状态推进和结果发布。Agent 服务负责语义判断和文本整合。检测模型服务负责执行图像级原子检测。不同角色之间的责任边界必须清晰，否则系统容易出现职责混乱。

例如，分类 Agent 可以判断需要执行哪些检测，但不应直接创建数据库子任务；Reasoning 服务可以执行模型并生成产物，但不应决定主任务是否完成；前端可以展示进度和结果，但不应根据局部状态自行推断数据库最终状态。本文将核心状态推进集中在 BIZ 服务中，使业务规则具有唯一执行位置。这一设计虽然增加了 BIZ 的复杂度，但有利于维护系统一致性。

表 3.1 业务角色与责任边界

角色	可以负责的内容	不应负责的内容
前端用户界面	用户交互、状态展示、结果查看、复核输入。	直接修改任务状态或绕过后端判断项目是否完成。
Core API	HTTP 接入、鉴权、WebSocket 推送。	复杂数据库事务和模型调度规则。
Core BIZ	业务状态机、任务创建、事务落库、事件发布。	直接执行深度学习模型或自由生成报告文本。
Activity 服务	Agent 调用、模型推理、能力执行和回调。	直接操作核心业务数据库。
Agent 平台	语义路由、单图总结、项目报告生成。	无约束地改变业务流程或访问内部状态。

表 3.1 表明，本文系统的复杂性并不只是功能数量多，而是每个功能都需要明确归属。尤其在 Agent 引入后，必须避免让 Agent 获得过大的业务权限。本文采用“Agent 输出建议，BIZ 执行业务”的方式，使智能能力和业务控制相互分离。

3.4 典型用例分析

为了进一步明确系统边界，可以将用户使用过程抽象为若干典型用例。第一个用例是“创建评估项目”。用户录入建筑名称、位置、建设年份、既有问题和评估目标，系统创建项目并进入基础信息阶段。第二个用例是“构建图像资产”。用户上传幕墙图像，并根据建筑立面或区域创建图像组，系统保存图像元数据和对象存储信息。第三个用例是“启动智能检测”。用户点击开始检测后，系统为每张已上传图像创建主任务，由 Worker 自主推进分类、检测和总结。第四个用例是“复核检测结果”。用户查看图像检测结果，对结果准确性进行判断并补充评论。第五个用例是“生成项目报告”。系统聚合项目上下文，调用报告 Agent 生成最终评估报告。

这些用例覆盖了从数据进入系统到决策结果输出的全过程。与普通后台管理系统相比，本系统的关键用例不是简单增删改查，而是长任务驱动的业务流程。尤其是智能检测用例，背后包含多个服务、多次回调和多种状态，必须通过状态机和异步编排保证可用。

3.5 功能需求分析

3.5.1 项目与权限需求

项目是所有业务数据的归属边界。系统需要支持用户注册、登录、令牌刷新、头像管理、项目创建、项目删除、项目列表查询和项目详情维护。所有项目相关接口都需要进行用户身份校验，保证用户只能访问自己的项目数据。项目阶段推进也需要校验，例如从智能检测阶段进入人工复核阶段时，必须保证项目下所有图像检测主任务均已成功。

3.5.2 图像资产需求

图像资产构建是后续 Agent 检测的基础。系统需要支持图像组管理、图像上传、图像移动、批量删除、批量移动、原图查看、上传状态展示和上传失败处理。由于图像可能较多，前端需要自适应卡片布局、分组折叠、批量选择和实时状态提示。后端需要通过对象存储保存原图，通过数据库保存元数据，通过 WebSocket 推送图像状态变化。

3.5.3 智能检测需求

智能检测阶段需要支持项目级启动检测、失败重试、状态查询和结果查询。启动检测时，系统应为所有已上传图像创建检测主任务。检测流程由 Worker 自主推进，用户不需要逐张图像操作。失败重试需要清理旧任务、旧检测产物和缓存，再重新创建任务。状态查询用于页面刷新后的状态恢复，WebSocket 用于页面运行时的实时更新。

3.5.4 复核与报告需求

复核阶段需要支持读取和保存图像检测任务的复核信息，包括准确性判断和补充评论。报告阶段需要支持报告状态查询和报告结果展示。报告生成本身属于后置任务，在用户推进项目阶段后自动触发。这样设计可以降低用户操作复杂度，使报告生成成为自然流程的一部分。

3.6 阶段化需求分析

为了使系统符合真实使用习惯，本文将项目流程拆分为五个阶段：项目基础信息、图像资产构建、Agent 智能检测、人工复核和报告生成。每个阶段既有独立功能，又为后续阶段提供数据。项目基础信息阶段提供建筑名称、地点、年份和描述等上下文；图像资产阶段提供图像和分组；智能检测阶段产生结构化结果和图像总结；人工复核阶段补充人的判断；报告生成阶段形成最终输出。

这种阶段化设计的意义在于把复杂评估任务拆分为用户可理解的步骤。用户不需要一次性理解所有数据对象，而是在流程推进中逐步完成输入、确认和输出。对于系统而言，阶段边界也提供了校验条件。例如，没有上传图像时不能进入检测阶段；检测任务未全部成功时不能进入人工复核阶段；报告生成完成后项目才能进入最终完成状态。

阶段化设计还使前端交互更清晰。图像资产阶段允许编辑、移动和删除图像；进入检测阶段后，资产编辑能力应逐步收敛；进入复核阶段后，用户关注点从数据准备转为结果确认；进入报告阶段后，用户关注点转为阅读和导出结论。不同阶段的按钮、权限和状态展示需要与业务含义一致。

3.7 非功能需求分析

3.7.1 可扩展性

建筑幕墙检测能力具有持续扩展需求。当前系统包含五类原子检测任务，但未来可能新增连接件松动、密封胶老化、幕墙变形、热工异常等检测能力。因此系统不能将检测流程写死在某个单体函数中，而应通过任务代码、统一接口和注册表机制组织能力。Agent 输出也应限制在系统已注册能力范围内，以便新增能力时只需补充任务代码、模型服务和数据库字段。

3.7.2 一致性

检测任务涉及多服务协作和异步回调。一旦状态更新顺序混乱，前端进度和数据库结果就可能不一致。本文采用“先建记录、后调服务、回调落库、事务推进”的原则。即在调用外部 Activity 服务前，BIZ 服务先创建任务记录并设置状态；Activity 服务只通过回调上报结果；BIZ 在事务中更新结果并判断是否推进主任务。该设计保证了系统状态由业务中心统一掌握。

3.7.3 可观测性

Agent 和模型调用存在外部依赖，失败原因可能来自网络、模型、对象存储、数据库或第三方 Agent 平台。因此系统需要完整日志和状态追踪。每个请求需要携带 request ID 并在 HTTP、gRPC 和回调链路中透传；每个任务需要记录 UUID、状态、开始时间、完成时间和运行耗时；每次 WebSocket 广播和 RocketMQ 事件也需要有日志。可观测性是长链路系统稳定运行的重要保障。

3.7.4 可部署性

毕业设计不是脚本演示，系统需要能够在服务器上通过容器方式启动。由于系统由多个服务组成，部署需要处理镜像构建、服务依赖、环境变量、消息队列 Topic 初始化、模型文件挂载和网络端口映射。尤其是 Reasoning 服务同时依赖 Go 运行时、Python 环境和模型权重，部署方案需要兼顾镜像体积和模型文件管理。本文采用服务镜像加模

型挂载的方式，使模型文件可以独立维护，避免镜像过大。

3.7.5 用户体验

用户体验需求主要体现在等待过程和结果解释上。检测任务耗时不可避免，因此前端需要以进度条、流程节点和图像蒙层展示状态。结果解释方面，用户需要同时看到原图、检测产物、结构化指标和 Agent 总结。对于区域类结果，系统需要支持在总体数据和各区域数据之间切换，使用户能够从总体结论追踪到具体区域。

3.8 关键挑战

本文系统实现并非简单 CRUD，而是在多个维度存在复杂性。第一，业务链路长。用户从上传图像到生成报告，需要经历资产管理、分类路由、并发检测、图像总结、人工复核和项目报告多个阶段。第二，服务协作复杂。系统包含前端、API、BIZ、三个 Activity 服务、数据库、对象存储、缓存和消息队列。第三，状态同步复杂。检测状态既要写入数据库，又要通过 RocketMQ 和 WebSocket 推送给前端，还要在页面刷新后可恢复。第四，结果结构复杂。不同检测任务输出字段不同，区域数量不定，产物图像数量不定，报告生成还需要将这些结果重新组织。

第五，报告输入组织复杂。项目级报告并不是简单拼接所有检测结果，而是需要把原子指标、图像总结、图像组信息和人工复核意见组织为模型可理解的上下文。上下文过短会导致报告缺少依据，上下文过长又可能造成重点丢失。因此，如何在工程系统中构造合适的中间语义层，是本文需要解决的重要问题。

第六，跨语言能力集成复杂。视觉检测模型主要运行在 Python 环境中，而后端业务服务采用 Go 实现。系统需要让 Python 专注模型推理，让 Go 负责下载、上传、回调和状态管理。跨语言边界既要简洁，又要保证错误能够被 Go 层捕获并反馈给上游状态机。

因此，本文的工作量主要不体现在某个局部算法，而体现在跨模块的工程链路设计和 Agent 能力嵌入。系统需要同时满足“智能性”和“工程稳定性”：Agent 负责语义判断和信息整合，工程系统负责协议、状态、存储和容错。二者缺一不可。

3.9 数据流分析

系统数据流从图像上传开始。前端获取上传授权后将图像上传至对象存储，BIZ 服务记录图像元数据。启动检测后，BIZ 读取图像信息并生成原图访问 URL，Classification 服务下载缩略图并调用 Agent 输出任务代码。分类结果回调后，BIZ 创建子任务，并为 Reasoning 服务生成原图访问 URL 和产物上传授权。Reasoning 服务执行模型，上传图像产物，将报告 JSON 和产物清单回调给 BIZ。BIZ 将报告字段解析落库，并在全部子任务完成后触发 Summary 服务生成单图总结。项目报告阶段，BIZ 聚合项目数据并交由 Summary 服务生成最终报告。

这个数据流说明系统中同时存在三类数据：文件数据、结构化数据和语义数据。文件数据包括原图和检测产物，存储在 MinIO；结构化数据包括任务状态、指标字段和区域 JSON，存储在 MySQL；语义数据包括单图总结和项目报告，由 Agent 生成后写入数据库。三类数据分别服务于不同目标：文件用于复核，结构化数据用于追踪和展示，语义数据用于报告和决策。

3.10 约束条件

系统设计还需要满足若干约束。第一，模型服务和业务服务应保持边界清晰。Python 模型不应直接操作数据库和对象存储业务 Key，否则后续新增模型会带来大量耦合。第二，Agent 输出必须可校验。分类结果需要解析为任务代码，报告结果需要通过固定接口回调，不能让自由文本直接控制状态。第三，检测任务需要支持失败状态和重试扩展。即使当前版本只实现基础重试，也必须在数据库设计和任务清理逻辑上为后续扩展预留空间。第四，前端状态需要能从服务端恢复，不能只依赖 WebSocket 增量消息，因为用户可能刷新页面或断线重连。

3.11 需求与设计目标映射

如表 3.2 所示，本文的设计目标并非孤立功能堆叠，而是围绕现实问题建立相应解决策略。每个策略都对应系统中的一个工程或 Agent 设计点，最终共同构成完整解决方案。

装

订

线

表 3.2 需求与设计目标映射

需求类型	现实问题	设计目标
图像资产管理 检测任务选择 长任务执行 多源结果整合	图像数量多、组织混乱、难以复核 不同材质需要不同检测能力 检测和总结耗时较长 底层指标难以直接形成报告	建立项目、图像组和图像三级资产结构。 使用分类 Agent 输出受控任务代码。 使用 Worker、异步启动和回调推进状态。 采用单图总结到项目报告的分层信息整合策略。
用户感知	长时间执行容易造成等待焦虑	使用 WebSocket、进度条和流程节点实时展示。
可追踪性	多服务链路失败难以定位	使用 UUID、状态机、日志和数据库记录追踪全流程。

4 面向幕墙评估的 Agent 自主调度与分层信息整合方法

4.1 方法概述

本文提出的核心方法可以概括为“受控 Agent 调度 + 原子检测工具化 + 分层信息整合”。其中，受控 Agent 调度用于解决检测前的任务选择问题，原子检测工具化用于解决不同模型能力的统一接入问题，分层信息整合用于解决检测结果到项目报告的语义转换问题。三者共同构成本文区别于普通工程系统的技术主线。

在该方法中，Agent 不是自由行动的黑箱，而是被放置在明确业务节点中。分类 Agent 只负责输出任务代码列表；图像级总结 Agent 只负责总结单张图像的检测结果；项目级报告 Agent 只负责融合项目级上下文。每个 Agent 的输入、输出和业务后果均由系统协议约束。这种设计既发挥大语言模型的语义能力，又保证工程系统可控。

本文方法的基本判断是：Agent 最适合承担“语义不确定但动作空间有限”的任务。幕墙图像检测正符合这一特点。图像中可能存在多种材料，材料判断具有一定语义不确定性；但系统可执行的检测工具只有五类，动作空间是有限的。项目报告生成同样如此：报告表达具有自然语言复杂性，但报告依据来自数据库和人工复核，输入边界是有限的。因此，本文把 Agent 放在可发挥语义能力的位置，同时用工程协议限制其行为范围。

4.2 Agent 在本文中的角色定义

为避免“只要接入大语言模型就是 Agent”的泛化理解，本文对 Agent 的角色进行明确限定。第一，Agent 不是聊天机器人。系统不提供开放问答入口，Agent 只在业务流程中的固定节点被调用。第二，Agent 不是数据库操作者。所有数据读取、落库和状态推进均由 BIZ 服务完成。第三，Agent 不是检测模型替代品。像素级缺陷识别仍由原子检测模型完成，Agent 主要负责路由和语义整合。

在此基础上，本文定义了三个 Agent 节点。分类 Agent 面向图像输入，输出任务代码；图像总结 Agent 面向单张图像的检测结果，输出面向用户的单图总结；项目报告 Agent 面向整个项目的汇总上下文，输出项目级报告。三个节点形成从“选择工具”到

“解释局部结果”再到“形成整体结论”的递进关系。

表 4.1 本文 Agent 节点设计

Agent 节点	输入	输出	业务作用
分类 Agent	原始图像与路由提示词。	检测任务代码列表。	决定是否执行五类原子检测。
图像总结 Agent	单图结构化检测结果。	单图自然语言总结。	压缩底层指标，形成图像级语义。
项目报告 Agent	项目上下文、单图总结、复核意见。	项目评估报告。	形成整体韧性评估结论。

表 4.1 中的三个节点构成本文的 Agent 主线。分类 Agent 解决“做什么检测”的问题，图像总结 Agent 解决“这一张图说明什么”的问题，项目报告 Agent 解决“这个项目整体说明什么”的问题。三者共同把图像检测从单点算法能力提升为项目级评估流程。

4.3 受控 Agent 调度机制

分类 Agent 的输入包括幕墙图像和路由提示词，输出为任务代码列表。任务代码限定为 corrosion、crack、stain、flatness 和 spalling 五类，分别对应金属锈蚀、石材裂缝、石材污渍、玻璃平整度和玻璃爆裂。Agent 需要根据图像中的材料和场景判断是否执行对应能力。如果图像不是幕墙，或无法判断需要检测的材料，则输出空列表。

受控调度机制的关键在于“让 Agent 判断，但不让 Agent 越界”。系统不允许 Agent 输出任意文本作为任务名称，也不允许 Agent 直接调用数据库或模型服务。Agent 只输出结构化任务代码，BIZ 服务负责解析、校验、创建子任务和调度 Reasoning 服务。这种方式将 Agent 的智能判断转化为工程系统可消费的有限动作。

在提示词设计上，系统要求 Agent 只输出合法 JSON，且任务代码必须来自固定集合。提示词会说明不同幕墙材料对应的检测能力：金属材料对应锈蚀检测，石材或混凝土材料对应裂缝和污渍检测，玻璃材料对应平整度和爆裂检测。这样，Agent 的视觉理解结果被映射到明确工具集合中。若 Agent 输出异常内容，服务端会解析失败并回调失败状态，而不会继续推进后续检测。

受控调度还包括输出后的二次校验。即使提示词要求 Agent 输出合法任务代码，服务端仍然需要对返回内容进行解析和过滤。只有 corrosion、crack、stain、flatness 和 spalling 五种代码能够被接受，重复代码会被去重，不支持的代码会被忽略或导致任务

失败。这样可以避免模型偶然输出解释性文字、中文任务名或未知字段时破坏状态机。

该机制的实质是把 Agent 的开放语义判断压缩为离散动作集合。对于工程系统而言，离散动作更容易落库、重试和展示；对于 Agent 而言，有限工具集合降低了决策难度。二者结合后，系统既不需要人工逐张选择检测能力，也不需要信任一个完全自由的模型代理。

4.4 为什么不是端到端大模型

一种看似简单的方案是让大语言模型直接读取图像并生成最终报告。本文没有采用该方案，原因有三点。第一，幕墙评估需要可追踪证据。用户需要查看原图、掩码、浮层、区域指标和置信度，而不是只接受一段不可追溯的自然语言。第二，项目级报告可能涉及大量图像和区域，大模型一次性处理会受到上下文长度和注意力分散限制。第三，工程系统需要可恢复状态。如果模型一步生成失败，很难判断失败发生在材质判断、缺陷识别还是报告融合阶段。

因此，本文采用分解式路线：图像理解中的任务选择交给分类 Agent，像素级和区域级事实交给专业检测工具，语义总结交给总结 Agent，流程状态交给工程系统。该路线虽然工程链路更长，但每一步职责更清晰，失败更容易定位，结果也更容易复核。

4.5 原子检测能力工具化

原子检测能力工具化是指将具体模型或算法封装为统一可调度能力。本文系统中，Reasoning 服务对 BIZ 只暴露一个启动接口，接口参数包括任务 UUID、任务代码、图像 UUID 和原图访问地址。Reasoning 服务内部根据任务代码路由到对应 Python 模型。模型执行后，Go 服务读取报告和产物，上传图像产物，并将结果回调给 BIZ。

这种封装方式有三个好处。第一，BIZ 服务不需要知道每个模型的运行细节，只需要知道任务代码。第二，Python 模型不需要知道业务数据库和对象存储细节，只需要按约定生成文件。第三，新增模型时可以沿用统一路由和回调机制。由此，检测能力从“脚本”变成了“可被 Agent 调度的工具”。

工具化还要求每个检测能力输出结构稳定。不同模型可以采用不同算法，但需要遵守统一的外部约定：报告字段应能落入数据库，图像产物应使用约定文件名，执行失

败应被 Go 服务捕获，成功或失败都必须通过回调上报。只有满足这些条件，检测能力才能成为 Agent 可调用工具，而不是只能人工运行的实验脚本。

在本文系统中，原子检测工具并不直接暴露给前端，也不由 Agent 直接调用。Agent 只输出任务代码，Worker 根据任务代码调用 Reasoning 服务，Reasoning 再路由到具体模型。这种间接调用链路看似更长，但能够保留业务控制权，避免 Agent 越权调用模型或绕过状态机。

4.6 检测任务状态流转

检测任务状态流转是方法落地的骨架。每张图像对应一个主任务，主任务从 pending 开始，Worker 消费后进入 classifying。分类成功后，如果 Agent 输出了需要执行的任务代码，系统创建对应子任务并进入 detecting；如果 Agent 输出空列表，说明该图像无需执行原子检测和总结，可以直接成功结束。子任务全部成功后，系统创建图像总结任务并进入 summarizing。图像总结成功后，主任务进入 succeeded。任一必要节点失败，主任务进入 failed。

这一状态机表达了系统对 Agent 决策的理解。Agent 的路由结果决定后续是否创建子任务，子任务结果决定是否进入总结，总结结果决定主任务是否完成。状态机既反映业务逻辑，也为前端进度展示提供数据基础。

表 4.2 检测任务状态含义

状态	业务含义
pending	主任务已创建，等待 Worker 消费。
classifying	分类 Agent 正在判断该图像需要执行哪些检测能力。
detecting	至少一个原子检测子任务已创建，系统正在等待子任务完成。
summarizing	原子检测已满足进入总结条件，图像总结 Agent 正在生成单图总结。
succeeded	该图像检测链路已成功结束。
failed	必要节点失败，当前图像检测链路失败。

状态机设计还需要处理一个容易被忽略的情况：分类 Agent 判断不需要执行任何检测。此时系统不应创建空的子任务，也不应调用图像总结 Agent，因为没有原子检测结果需要总结。该主任务应直接进入成功状态。这一规则体现了 Agent 路由结果对流程结构的影响，也说明状态机必须理解“无需检测”与“检测失败”之间的差别。

4.7 分层信息整合策略

分层信息整合策略是本文最重要的创新设计之一。其基本思想是：不把所有检测结果一次性交给项目报告 Agent，而是先在图像级进行总结，再在项目级进行综合。该策略可表示为三层结构：

（1）原子事实层：由五类检测模型输出结构化结果，包括缺陷是否存在、区域数量、置信度、像素占比、区域坐标和产物图像。

（2）图像语义层：由图像级总结 Agent 将一张图像的多个原子结果压缩为自然语言总结，表达该图像的主要问题和维护建议。

（3）项目报告层：由项目级报告 Agent 汇总所有图像总结、项目背景、图像组信息和人工复核意见，生成项目级韧性评估报告。

这种层次化组织方式符合工程认知过程。检测人员通常也不会直接根据所有像素指标编写项目报告，而是先理解每张图像的主要情况，再总结整个建筑的整体风险。本文将这一人工认知过程固化为系统方法，使 Agent 报告生成具有更清晰的数据来源和推理层次。

分层信息整合也可以理解为一种面向工程报告的上下文管理机制。原子事实层保留最大信息量，但不适合直接给最终用户阅读；图像语义层舍弃过细字段，保留该图像的主要问题；项目报告层进一步聚合多张图像之间的共性和差异。三层之间不是简单摘要关系，而是从局部事实到局部判断，再到整体结论的逐级抽象。

与普通摘要不同，本文的分层整合具有明确的工程约束。图像级总结必须以结构化检测结果为依据，不能凭空推断；项目级报告必须考虑图像组、人工复核和项目背景，不能只罗列每张图像的结论。也就是说，每一层的输入都有边界，每一层的输出都有用途。分层信息整合不是为了减少字数，而是为了让信息从算法语义逐步转化为工程语义。

该策略还降低了项目报告 Agent 的认知负担。假设一个项目包含数十张图像，每张图像包含多个检测区域，如果直接输入全部区域指标，模型需要同时处理大量低层数据。采用图像级总结后，项目报告 Agent 面对的是“每张图的主要问题和建议”，更容易分析区域分布、共性问题和维护优先级。这种方法使 Agent 的上下文使用更接近人类

工程师撰写报告时的工作方式。

4.8 单图总结设计

单图总结的输入是某张图像的全部检测结果。对于只执行一个子任务的图像，总结重点是该任务的结论和建议；对于执行多个子任务的图像，总结需要综合不同缺陷之间的关系。例如，一张石材幕墙图像可能同时执行裂缝和污渍检测，单图总结需要说明是否存在裂缝、是否存在污渍、哪个问题更突出以及后续维护建议。

单图总结并不替代结构化数据。结构化数据仍然保存在数据库中，供前端展示和追溯；单图总结则作为更高层语义表示，供项目报告 Agent 使用。这样设计避免了在项目报告阶段重复处理大量底层字段，也避免了丢失原始检测指标。

4.9 项目报告生成设计

项目报告生成的输入包括项目基础信息、图像组信息、每张图像的检测状态、图像级总结、人工复核结果和用户补充评论。报告 Agent 需要在这些信息基础上输出结构化报告。报告不只是列出缺陷，而是需要说明主要风险、区域分布、维护优先级和总体结论。

项目报告阶段是本文方法的最终输出。它体现了 Agent 从局部信息到整体结论的综合能力。由于输入已经经过图像级总结压缩，项目报告 Agent 可以更关注跨图像和跨区域关系，而不是被区域坐标和像素统计干扰。这是分层信息整合策略相对直接拼接方案的核心优势。

在项目报告输入组织中，图像组名称具有特殊价值。用户可能将图像组命名为“北立面”“东侧裙楼”“玻璃幕墙区域”等，这些名称本身包含空间或业务语义。项目报告 Agent 可以利用这些分组信息分析不同区域的问题分布。与完全无结构的图像列表相比，按项目、图像组和图像三级组织输入，更有利于生成具有工程语义的报告。

4.10 人机协同机制

完全自动化评估在工程领域并不总是可靠，尤其是在模型置信度不足或图像质量较差时。因此本文系统引入人工复核阶段。用户可以查看检测产物和结构化指标，对结

果标记准确或不准确，并填写补充评论。这些复核信息不会改变原子检测事实，但会作为项目报告生成的补充上下文。

该设计体现了人机协同思想。Agent 和模型承担批量处理与初步分析，人类保留最终判断和补充解释能力。系统通过复核字段将人工意见结构化保存，使项目报告既包含自动检测结果，也包含人的专业判断。

人工复核并不是对自动化能力的削弱，而是工程系统可信性的组成部分。幕墙评估涉及维护成本和安全责任，系统应允许用户对检测结果进行确认或纠正。本文将复核信息作为项目报告输入，而不是直接覆盖模型结果，这样既保留自动检测证据，也保留人工判断痕迹。最终报告因此能够体现“模型初判 + 人工确认 + Agent 汇总”的协同过程。

4.11 方法优势分析

与固定检测流程相比，本文方法能够减少无意义模型调用，并使检测任务更符合图像内容。与直接报告生成相比，分层信息整合能够降低上下文压力，提高报告生成稳定性。与单纯工程系统相比，Agent 的引入使系统具备语义判断和自然语言整合能力。与开放式 Agent 相比，受控输出和工程状态机保证了系统边界和可追踪性。

因此，本文方法并不是简单把大语言模型接入系统，而是围绕幕墙韧性评估业务重新组织 Agent 的角色。Agent 被放在最能体现价值的三个节点上：检测前路由、检测后压缩、项目级融合。这种位置选择使 Agent 能力和工程流程形成互补关系。

4.12 方法流程小结

从流程上看，本文方法可以概括为：用户启动项目检测，BIZ 创建主任务，Classification Agent 输出任务路线，BIZ 创建对应子任务，Reasoning 服务执行原子检测，BIZ 汇总子任务结果，Summary 服务生成单图总结，项目进入人工复核，最后 Summary 服务生成项目报告。每一步都有明确输入、输出和状态变化。

这种流程具有较强可扩展性。新增检测能力时，需要新增任务代码、模型实现、结果结构和展示逻辑，但分类 Agent 和 Worker 调度框架可以复用。新增报告维度时，可以调整项目级报告 Agent 的提示词和输出结构，而无需改变原子检测链路。由此，本文

方法不仅适用于当前五类幕墙检测任务，也可扩展到更多建筑运维智能评估场景。

装
订
线

5 支撑 Agent 落地的系统架构与工程实现

5.1 总体架构设计

本文系统采用前后端分离和服务化架构。前端负责用户交互、阶段流程、图像展示、检测进度和报告页面；后端按照职责拆分为 Core API、Core BIZ、Activity Classification、Activity Reasoning、Activity Summary 和 ICW Common 等模块。基础设施包括 MySQL、Redis、MinIO 和 RocketMQ。这样的拆分并不是为了追求形式上的微服务，而是为了将不同性质的工作隔离：HTTP 接入、业务状态、Agent 调用、模型推理、报告生成和共享协议分别由不同模块承担。

Core API 是前端入口，提供 HTTP 和 WebSocket 能力。Core BIZ 是业务中心，负责数据库事务、对象存储、任务 Worker、回调处理和事件发布。Activity Classification 封装分类 Agent，Activity Reasoning 封装 Python 检测模型，Activity Summary 封装图像总结和项目报告 Agent。ICW Common 统一提供 Protocol Buffers 接口、枚举定义、gRPC 调用封装、日志工具、错误结构和通用工具函数。

这种架构的核心原则是“智能能力外置，业务状态内聚”。Agent 和模型能力被放在 Activity 服务中，可以独立演进；项目、图像、任务和报告状态集中在 BIZ 服务中，避免状态分散。API 服务不承担复杂业务，只负责把前端请求转换为 BIZ 调用，并把 BIZ 产生的事件推送给前端。这样的职责划分使系统既能接入外部智能能力，又能保持核心业务一致性。

从工作量角度看，本文系统不是简单调用一次大语言模型，而是需要把多个异构能力编排在同一业务流程中。Classification 调用的是外部 Agent 平台，Reasoning 调用的是本地 Python 模型，Summary 再次调用外部 Agent 平台，三类能力的运行时间、失败模式和返回格式均不同。系统必须通过统一协议和状态机把它们组织起来。

5.2 通信与协议设计

系统外部通信采用 HTTP 和 WebSocket，内部服务通信采用 gRPC。前端通过 HTTP 发起项目、图像、检测、复核和报告相关请求；API 服务再通过 gRPC 调用 BIZ 服务。

表 5.1 系统服务划分

服务或模块	主要职责
Core Web	项目流程页面、图像资产管理、检测进度可视化、复核与报告展示。
Core API	HTTP 接口、鉴权入口、WebSocket 连接、RocketMQ 消息消费与前端广播。
Core BIZ	业务状态机、数据库事务、对象存储封装、检测 Worker、回调落库和事件生产。
Activity Classification	调用分类 Agent，输出需要执行的检测任务代码。
Activity Reasoning	调用五类原子检测模型，上传检测产物并回调检测结果。
Activity Summary	调用总结 Agent，生成单图总结和项目级评估报告。
ICW Common	共享协议、枚举、gRPC 基础设施、日志、错误和工具函数。

BIZ 调用 Activity 服务启动分类、推理和总结任务，Activity 服务执行完成后通过 gRPC 回调 BIZ。BIZ 将状态变化写入数据库并发布 RocketMQ 事件，API 服务消费事件后通过 WebSocket 推送给前端。

gRPC 的引入使服务间接口具有明确契约。系统中的任务状态、项目进度、检测任务代码、复核结论和报告状态均通过共享枚举定义，避免不同服务各自维护字符串常量。请求 ID 通过 gRPC metadata 透传，使 HTTP 请求、BIZ 处理、Activity 调用和回调日志能够关联起来。这对于复杂链路调试非常重要。

协议设计还承担了约束 Agent 输出的作用。分类结果、推理结果和总结结果都通过固定 gRPC 回调接口进入 BIZ。这样，即使 Agent 或模型服务内部实现发生变化，上层业务仍然只依赖稳定协议。接口定义统一放在 Common 模块中，并通过代码生成供 API、BIZ 和 Activity 服务使用，减少手写结构体转换带来的错误。

在接口设计中，本文尽量避免让调用方依赖被调用方内部结构。例如，Reasoning 服务不要求 BIZ 预先知道污渍和平整度任务会生成多少区域图像；BIZ 只提供受限上传前缀，Reasoning 执行后回调实际产物清单。再如，Summary 服务不直接读取数据库，而是接收 BIZ 生成的数据源或压缩上下文。这样，每个服务只暴露必要边界，内部实现可以独立变化。

5.3 共享协议与枚举治理

由于系统跨越 API、BIZ、Classification、Reasoning 和 Summary 多个 Go 仓库，如果每个仓库都自行定义请求结构、状态枚举和错误常量，很容易出现字段名不一致、枚举值不一致和转换函数膨胀等问题。本文将公共协议收敛到 ICW Common 模块，并通过 Protocol Buffers 生成代码。所有服务都依赖生成后的结构体和枚举，减少手写 DTO

转换。

共享协议治理在项目中体现为三个方面。第一，业务状态使用统一枚举，例如项目进度、图像状态、检测任务状态、子任务状态、报告状态等。第二，RPC 接口由 proto 文件定义，服务端实现接口，客户端调用生成代码。第三，API 层返回给前端的结构也尽量复用生成结构，减少 API 与 BIZ 之间重复定义。这样做提高了开发成本的前期投入，但降低了后续维护成本。

在 Agent 场景中，枚举治理尤其重要。分类 Agent 的输出最终会被解析为检测任务代码，如果代码在不同服务中以字符串散落维护，新增或修改任务时很容易遗漏。通过共享枚举和解析函数，系统可以在编译阶段暴露部分问题，并在运行时统一校验非法值。

5.4 数据库与状态机实现

数据库设计围绕项目、图像资产和检测任务展开。项目表保存建筑基础信息和项目进度；图像组和图像表保存资产组织关系；检测主任务表保存每张图像的检测状态；五张子任务表分别保存不同原子检测能力的结构化结果；图像总结表保存单图总结；项目报告表保存项目级报告结果。该设计虽然表数量较多，但可以将不同任务的专属指标清晰拆分，避免所有结果塞入一个无法维护的大字段。

检测主任务状态机是系统复杂度的集中体现。主任务从 pending 开始，经过 classifying、detecting、summarizing，最终进入 succeeded 或 failed。分类结果决定是否创建子任务，子任务结果决定是否进入总结，总结结果决定主任务是否完成。所有状态推进均由 BIZ 控制，Activity 服务只上报结果。这种设计避免外部模型服务直接操作核心数据库，提高系统一致性。

五类子任务表采用“通用字段 + 专属指标”的设计。通用字段包括任务 UUID、主任务 ID、用户 ID、项目 ID、图像 ID、状态、开始时间、完成时间和运行耗时；专属指标则根据检测任务差异定义。例如锈蚀检测需要记录锈蚀区域数量、锈蚀像素和锈蚀占比；平整度检测需要记录平整、不平整或非玻璃结论；爆裂检测只需要记录是否爆裂和置信度。相比所有结果统一存 JSON，这种设计便于后续查询、筛选和前端结构化展示。

数据库事务用于保证关键状态推进的原子性。例如，分类回调成功后，系统需要同时更新主任务路由字段、创建子任务、关联子任务 ID 并推进主任务到 `detecting`。如果这些操作分散执行，可能出现主任务已进入 `detecting` 但子任务尚未创建的中间状态。类似地，子任务回调时，系统需要在同一事务中写入检测结果、更新子任务状态、检查同一主任务下其他子任务状态，并在全部成功后创建总结任务。这些事务边界是系统一致性的核心。

数据库还承担了重试能力的基础。虽然本文当前阶段不展开单个子任务重试，但表结构和任务 UUID 已经为重试预留空间。失败任务可以通过主任务和子任务记录定位历史结果，后续可按图像维度删除旧子任务、清理对象存储并重新创建任务。没有结构化任务表，重试只能依赖文件状态或日志，难以成为可靠业务能力。

5.5 Detection Worker 实现

Detection Worker 是检测流程的编排核心。用户启动检测时，BIZ 服务先为项目下图像创建主任务并返回，Worker 随后按配置的最大并发数消费任务。每个任务执行时，Worker 先将主任务推进到分类阶段，调用 Classification 服务；分类回调成功后，Worker 根据任务代码创建子任务并调用 Reasoning 服务；所有需要执行的子任务成功后，Worker 创建图像总结任务并调用 Summary 服务。

Worker 设计解决了同步调用难以支撑长任务的问题。如果前端请求一直等待所有模型执行结束，用户体验和服务稳定性都会受到影响。通过 Worker，启动检测接口只负责创建任务和入队，真正的检测链路在后台推进。前端通过 WebSocket 获得状态变化，从而实现“请求立即返回、任务持续执行、进度实时可见”的体验。

Worker 还负责处理状态之间的因果关系。例如，分类失败时不应继续创建子任务；分类成功但没有任何检测能力需要执行时，应直接完成主任务；任一必要子任务失败时，主任务最终应进入失败；全部子任务成功后，才能创建图像总结任务。这些逻辑如果分散在各 Activity 服务中，会导致状态难以维护。集中在 Worker 中处理，可以保持业务规则一致。

Worker 的另一个职责是并发控制。检测任务可能同时包含多张图像，如果不限限制并发，分类 Agent、Python 模型、对象存储和数据库都会承受突发压力。系统通过最大

并发配置限制后台任务推进速度，使吞吐量和资源占用保持平衡。并发控制不是简单性能参数，而是保证外部 Agent 平台、模型服务和存储服务稳定运行的必要条件。

Worker 与回调之间形成非阻塞协作关系。Worker 调用 Activity 服务时，只要求对方成功接收任务；真正执行结果通过回调返回。这种方式避免 BIZ Worker 长时间阻塞等待模型执行，也使 Activity 服务可以在内部使用协程和并发队列。对于外部 Agent 调用和深度学习推理这种长耗时任务，回调式架构比同步阻塞更适合。

5.6 对象存储与产物管理

幕墙图像和检测产物体积较大，不适合直接存入数据库。系统使用 MinIO 作为对象存储，数据库只保存元数据和对象 Key。图像上传时，BIZ 生成预签名上传 URL，前端直接上传对象存储。检测产物上传时，由于污渍和平整度任务可能产生不定数量的区域图像，系统采用受限前缀 POST Policy，允许 Reasoning 服务在指定前缀下上传动态数量的 PNG 产物。

对象访问 URL 具有有效期，系统使用 Redis 缓存预签名下载 URL，降低重复生成成本。删除对象时，系统先删除 Redis 缓存，再删除 MinIO 对象，避免前端使用已失效缓存。对于检测产物，Reasoning 服务还计算 SHA256 并回调给 BIZ 保存，用于后续校验和追踪。

产物管理的难点在于不同检测任务的产物数量不同。裂缝检测通常输出原图、掩码图和浮层图；污渍和平整度检测可能根据区域数量输出多个编号图像；爆裂检测可能只有原图和报告。系统不能要求 BIZ 在调用前预知所有产物名称，因此采用前缀授权和回调产物清单的方式。Reasoning 服务实际发现并上传哪些文件，就在回调中告诉 BIZ。这样既支持动态产物数量，也避免业务服务和模型脚本强耦合。

对象存储设计还影响前端展示。检测结果页面需要根据任务类型展示不同产物组合：锈蚀任务展示标注图，裂缝任务展示浮层图和掩码图，污渍任务展示总体标注图以及每个区域的裁剪图和浮层图，平整度任务展示区域、线条、梯度和频域图。由于产物最终都以对象方式存储，前端只需要通过 BIZ 返回的预签名访问地址展示图像，而不需要理解对象存储内部权限机制。

5.7 Reasoning 服务实现

Reasoning 服务是 Go 与 Python 模型能力的连接层。Go 服务负责 gRPC 接口、请求校验、运行目录准备、原图下载、Python Worker 调用、产物上传和回调。Python Worker 负责加载和执行具体检测模块。为避免每次检测都重新启动 Python 脚本和加载模型，系统采用常驻 Worker 和懒加载策略：某类模型首次调用时加载，后续任务复用已加载模型。

该实现兼顾了 Go 的工程能力和 Python 的模型生态。Go 适合处理网络、并发、对象存储和服务通信；Python 适合调用深度学习模型和图像处理库。通过明确边界，系统避免在 Python 中实现业务上传和数据库逻辑，也避免在 Go 中直接处理复杂模型推理。

Reasoning 服务的实现经历了从脚本执行到常驻 Worker 的优化。若每次任务都通过命令行启动 Python 脚本，则模型加载成本会重复发生，检测延迟较高。采用常驻 Python Worker 后，Go 服务通过标准输入输出或内部协议向 Worker 发送任务，Worker 在首次使用某个模型时加载并缓存。这样既保留 Python 模型实现，又降低重复加载成本。

运行目录设计也体现了工程约束。每次任务按任务代码和图像 UUID 创建独立目录，执行前清空旧目录并重新下载原图，避免重试或重复执行时读取历史产物。Python 模型只需要在约定目录中生成约定文件，Go 服务负责读取、上传和清理。这种目录约定比共享临时文件名更可靠，也便于调试和定位。

5.8 前端可视化实现

前端按照项目阶段组织页面。图像资产阶段提供上传、分组、批量操作和原图查看；智能检测阶段提供开始检测、失败重试、状态统计、实时进度和检测结果查看；人工复核阶段复用检测结果视图，并增加评论和准确性选择；报告阶段展示项目级评估报告。

检测进度可视化由两部分组成。一部分是图像卡片上的整体进度条，用于快速展示每张图像当前状态；另一部分是进度弹窗中的流程图，用于展示分类、并行原子检测、总结和结束节点。检测完成后，结果弹窗展示原始信息、子任务结果和图像总结。对于不同检测任务，系统根据产物数量自适应展示单图、双图或四图布局。

前端状态合并也是实现难点之一。WebSocket 消息可能因为网络、刷新或并发而出

现延迟，前端不能简单用最后一条消息覆盖全部状态。系统因此同时提供状态查询接口和增量消息合并逻辑。页面首次进入或刷新时通过查询接口获得全量状态，运行过程中再用 WebSocket 消息更新对应图像和节点。这种设计提高了实时展示的可靠性。

检测结果展示采用任务类型驱动布局。不同原子任务的数据字段和图像产物不同，前端需要将数据库中的结构化结果转化为用户可读界面。例如，裂缝和污渍需要展示区域列表，平整度需要展示平整、不平整或非玻璃结果，爆裂任务没有区域数据。前端并非只显示 JSON，而是将字段翻译为中文指标，并根据区域序号切换对应产物。这样的展示方式使复杂检测数据能够被普通用户理解。

人工复核阶段复用检测结果视图，并在弹窗中增加评论和准确性选择。该设计避免重复开发两套结果页面，也保持用户体验一致。报告阶段则从检测图像视图转向项目报告视图，体现流程从图像级事实逐步走向项目级结论。

5.9 部署实现

系统最终以 Docker 镜像和 Docker Compose 方式部署。Go 服务镜像尽量保持轻量，Reasoning 服务由于需要 Python 环境和模型文件，采用 Go+Python 运行镜像并通过挂载目录提供模型权重。部署顺序上，先启动 RocketMQ，再初始化 Topic，随后启动三个 Activity 服务，再启动 BIZ，最后启动 API。这样的顺序保证依赖服务可用后，上层服务再开始接收请求。

部署过程中还需要处理服务地址、消息队列 NameServer、对象存储端点、Redis、MySQL 和 Agent 平台 Token 等环境配置。由于系统服务较多，部署配置本身也是工程工作量的一部分。本文通过显式环境变量和统一 Docker Compose 文件降低部署复杂度。

部署实践中还需要处理 RocketMQ Topic 初始化、Reasoning 模型文件挂载、Docker 镜像体积和服务启动顺序等问题。Reasoning 服务包含模型依赖，不适合作为纯 Go scratch 镜像；若把模型权重直接打入多架构镜像，镜像体积和推送稳定性都会受到影响。因此本文采用运行镜像与模型目录分离的方式，在服务器上通过只读挂载提供模型文件。该方案更适合大模型文件和检测权重的长期维护。

5.10 工程复杂度总结

从表面看，系统使用的是成熟工程技术；但从整体链路看，工作量集中在多服务协同和状态一致性上。用户一次点击“开始检测”，背后会触发主任务创建、Worker 入队、分类 Agent 调用、分类回调、子任务创建、多个推理任务并发执行、产物上传、子任务结果落库、图像总结、报告准备、WebSocket 推送和前端状态合并等操作。每个环节都需要处理失败、重试、日志和数据一致性。

因此，本文工程实现不是简单页面和接口堆叠，而是为了支撑 Agent 自主调度和分层信息整合而构建的复杂运行环境。该环境使 Agent 能力从一次性调用转化为可持续执行的业务流程。

表 5.2 一次图像检测任务涉及的主要工程环节

环节	说明
任务创建	为图像创建主任务，记录用户、项目、图像和初始状态。
分类路由	调用分类 Agent，解析任务代码并校验输出合法性。
子任务生成	根据路由结果创建对应子任务，并关联到主任务。
原子检测	调用 Reasoning 服务执行模型，生成报告和图像产物。
产物上传	使用对象存储授权上传动态数量产物，并记录产物清单。
结果落库	将不同任务的结构化字段写入对应子任务表。
状态推进	判断子任务聚合状态，决定是否进入图像总结或失败终态。
实时通知	通过 RocketMQ 和 WebSocket 将状态变化推送给前端。
图像总结	调用 Summary 服务生成单图总结并写入数据库。

表 5.2 展示了单张图像检测背后的主要工程环节。对于一个包含多张图像的项目，上述链路会并发执行多次，因此系统必须在并发控制、事务处理和前端状态合并上保持一致。这也是本文工程工作量的重要体现。

6 系统验证与应用分析

6.1 验证目标

本文系统验证的重点不是单个检测模型的精度对比，而是验证所提出的 Agent 调度与分层信息整合方案能否在完整工程链路中运行。具体来说，需要验证系统是否能够完成项目级图像资产管理，是否能够由 Agent 输出检测路由，是否能够自动调度原子检测能力，是否能够实时展示检测进度，是否能够生成单图总结和项目报告，以及系统在部署环境中是否能够稳定启动和协同运行。

围绕上述目标，验证工作分为功能验证、链路验证、状态验证、交互验证和部署验证五个方面。功能验证关注用户能否完成流程；链路验证关注服务之间是否能够正确调用和回调；状态验证关注数据库和前端状态是否一致；交互验证关注用户能否理解检测进度和结果；部署验证关注 Docker 环境下服务是否能够正常运行。

6.2 功能验证

功能验证覆盖项目全流程。首先，用户可以完成注册、登录、项目创建和基础信息维护。其次，用户可以在图像资产阶段创建图像组、上传图像、移动图像、查看原图和进行批量操作。再次，用户可以进入 Agent 智能检测阶段，点击开始检测后系统为所有已上传图像创建任务，并进入后台执行流程。检测过程中，用户可以看到图像级进度和节点级状态。检测完成后，用户可以打开结果弹窗查看原始信息、各子任务结果、图像产物和图像总结。

在人工复核阶段，用户可以查看已经完成的检测结果，并为每张图像补充评论或准确性判断。在报告生成阶段，系统能够触发项目级报告生成，并在报告完成后展示结果。上述验证说明系统实现了从图像资产到项目报告的完整闭环，而不是只完成单个接口或单个模型调用。

表 6.1 展示了系统主要验证项。与普通表单系统不同，本系统每个功能项往往涉及多个服务。例如智能检测并不是单个按钮触发单个接口，而是关联任务创建、Agent 路由、原子检测、回调落库、状态推送和前端合并。因此，功能验证也必须从端到端链路

表 6.1 主要功能验证项

验证项	验证内容	预期结果
项目基础信息	创建项目、编辑建筑信息、推进阶段	项目状态正确保存并可恢复。
图像资产	上传、分组、移动、删除、查看原图	图像状态和元数据正确展示。
智能检测	启动检测、路由、推理、总结	主任务按状态机推进。
实时进度	WebSocket 接收状态变化	前端进度条和节点同步更新。
人工复核	保存评论和准确性判断	复核信息写入数据库并参与报告上下文。
项目报告	触发生成、状态推送、报告展示	报告完成后可在前端查看。

角度进行。

6.3 Agent 调度验证

Agent 调度验证关注分类 Agent 的输出是否能够被工程系统正确消费。分类 Agent 输出任务代码列表后，BIZ 服务根据任务代码设置主任务中的执行标记，并创建对应子任务。如果 Agent 输出空列表，系统不会创建任何子任务和图像总结任务，而是直接将主任务推进到成功状态。该逻辑保证系统不会对无关图像执行无意义检测。

在实际链路中，分类 Agent 的输出会影响后续所有阶段。若输出 crack 和 stain，系统只创建裂缝和污渍子任务；若输出 flatness 和 spalling，系统只创建玻璃平整度和玻璃爆裂子任务。前端展示也会根据路由结果决定选项卡和进度节点。这说明 Agent 输出不仅是文本结果，而是真正参与了业务流程控制。

6.4 分层信息整合验证

分层信息整合验证关注单图总结和项目报告是否形成上下游关系。原子检测完成后，系统将结构化结果整理为单图总结输入，Summary 服务生成图像级总结并回调 BIZ 保存。项目报告生成时，系统不再直接把所有底层区域 JSON 无差别输入模型，而是汇总项目基础信息、图像分组、单图总结和复核信息。

该验证表明，系统将报告生成拆成了两个语义层次。图像级总结解决“这张图像说明什么”，项目级报告解决“整个项目说明什么”。这种分层方式使报告生成更符合工程认知路径，也为后续优化 Agent 提示词、接入知识库和专家评审提供了清晰位置。

为了验证分层信息整合的必要性，可以从信息规模和信息质量两个角度分析。信息规模方面，原子检测结果包含大量区域指标，如果项目图像数量增加，原始 JSON 长

度会线性甚至更快增长。图像级总结将每张图像压缩为较短语义描述，使项目报告输入规模更可控。信息质量方面，单图总结已经完成局部判断，项目报告 Agent 可以更关注跨图像模式和整体建议，而不是重复解释底层指标。

6.5 状态一致性验证

状态一致性是系统可靠性的关键。检测任务状态同时存在于数据库、后端 Worker、WebSocket 消息和前端状态中。如果任一环节处理不当，用户可能看到错误进度。系统通过主任务状态、子任务状态和节点状态共同表达进度，并在页面刷新时通过查询接口恢复当前状态，在实时运行时通过 WebSocket 增量更新。

系统验证中重点检查了几类状态：分类失败后主任务是否进入失败，子任务失败后主任务是否最终失败，所有子任务成功后是否进入总结，总结成功后主任务是否成功，项目推进时是否校验所有任务成功。这些状态验证确保用户看到的进度和数据库中保存的真实状态一致。

6.6 部署验证

部署验证采用 Docker Compose 完成。系统需要先启动 RocketMQ NameServer 和 Broker，并创建项目事件 Topic；随后启动 Classification、Reasoning 和 Summary 服务；再启动 BIZ；最后启动 API。Reasoning 服务通过挂载模型目录读取权重文件，避免将大模型文件直接打入镜像导致镜像过大。BIZ 和 API 通过环境变量连接数据库、Redis、MinIO 和 RocketMQ。

部署验证过程中重点检查服务启动顺序、消息队列 Topic 初始化、gRPC 地址配置、对象存储访问、模型目录挂载和 WebSocket 消息推送。由于系统依赖较多，部署验证能够反映真实工程复杂度。最终系统能够在容器环境下完成服务启动和链路调用，说明系统具备进一步工程化的基础。

部署过程中也暴露出若干典型工程问题。例如，消息队列 Topic 需要在业务服务启动前完成初始化，否则 API 服务消费事件时会找不到路由；Reasoning 服务不能简单构建为纯 Go 镜像，因为它依赖 Python 运行时和模型文件；模型文件直接打入镜像会导致镜像体积过大，因此最终采用模型目录挂载方式。这些问题说明，智能系统的工程落

地不仅包含代码开发，还包含运行环境、依赖管理和服务编排设计。

6.7 应用价值分析

从应用价值看，系统解决了建筑幕墙评估中的几个实际问题。首先，图像资产管理使大量巡检图像能够按项目和图像组组织，便于后续追踪。其次，Agent 路由减少了人工判断检测能力的成本，使系统能够根据图像内容动态执行任务。再次，分层信息整合使检测结果能够逐步转化为可读报告，降低人工整理报告的负担。最后，实时进度和结果可视化提升了用户对长任务的可理解性。

与单一检测工具相比，本文系统更接近实际运维软件。它不仅告诉用户某张图是否有缺陷，还把图像、检测、总结、复核和报告纳入同一个流程。与纯 Agent 应用相比，本文系统具有更强工程约束，Agent 的输出会被数据库、状态机和接口协议管理。这种结合使系统既具有智能性，也具有可落地性。

表 6.2 不同方案对比

方案	优势	不足	本文改进
人工巡检	专家经验强，判断灵活	效率低、主观性强、难以规模化	用系统管理图像与流程，减少人工整理成本。
单模型检测	对单类缺陷有效	缺少材质路由和报告生成	将模型封装为 Agent 可调度工具。
直接大模型报告	实现简单，生成自然语言快	缺少证据追踪，输入过长不稳定	采用分层信息整合和结构化落库。
本文方案	可调度、可追踪、可复核、可报告	系统复杂度更高	通过工程链路承担复杂性，提升实际可用性。

如表 6.2 所示，本文方案并不是在所有方面都最简单，而是在实际业务价值和系统可控性之间取得平衡。相比人工巡检，它提高自动化程度；相比单模型检测，它形成评估流程；相比直接大模型报告，它保留结构化证据和工程状态。

6.8 局限性分析

当前验证仍存在局限。第一，原子检测模型主要用于系统链路验证，尚未在大规模真实幕墙数据集上进行严格精度评估。第二，Agent 总结和报告质量仍依赖提示词和外部模型能力，尚未建立专家评分体系。第三，系统部署依赖外部数据库、对象存储和 Agent 平台，生产环境还需要完善监控、权限隔离和数据备份。第四，项目报告目前主

要基于图像检测信息，尚未与 BIM、传感器和历史维修记录等更多数据源融合。

尽管如此，当前验证已经证明本文技术路线具备可行性。系统能够把 Agent 调度、原子检测、分层总结和工程状态机连接起来，形成一个完整可运行的软件原型。

6.9 本章小结

本章从功能、Agent 调度、分层信息整合、状态一致性、部署和应用价值等角度验证了系统。验证结果表明，本文系统能够完成从图像资产到项目报告的闭环流程，Agent 路由能够真实影响后续检测任务，分层信息整合能够降低报告生成输入复杂度，工程架构能够支撑长任务异步执行和实时展示。虽然系统仍需在模型精度和报告质量方面继续完善，但作为毕业设计原型，已经体现出完整工作量和明确技术特色。

7 总结与展望

7.1 全文总结

本文围绕建筑幕墙韧性评估中的现实痛点，设计并实现了一套基于 AI Agent 的软件系统。本文工作的重点不是重新提出某个检测模型，而是解决现实工程中更关键的链路问题：如何把海量幕墙图像、不同原子检测能力、人工复核信息和项目级报告生成组织为一个完整、可追踪、可部署的系统，并通过 Agent 提升任务调度和信息整合能力。

全文首先分析了建筑幕墙巡检从人工经验向智能评估转型过程中面临的痛点，指出现有单点检测方法难以解决项目级图像管理和报告生成问题。随后，本文梳理了建筑立面检测、计算机视觉和 AI Agent 相关研究，说明 Agent 工具调用和多阶段工作流为幕墙评估提供了新的技术机会。在问题建模基础上，本文提出受控 Agent 调度和分层信息整合策略，将分类 Agent、图像级总结 Agent 和项目级报告 Agent 分别嵌入检测前、检测后和项目报告阶段。最后，本文完成了包含前端、API、BIZ、Activity 服务、数据库、对象存储、消息队列和部署配置的系统原型。

7.2 主要成果

本文取得的主要成果包括以下三点。第一，设计了面向多材质幕墙图像的 Agent 路由机制，使系统能够根据图像内容动态决定需要执行的检测能力。该机制避免了固定执行全部模型的低效方式，也使检测流程更符合幕墙材料差异。

第二，提出并实现了分层信息整合策略。系统先保存原子检测结构化结果，再生成图像级总结，最后生成项目级报告。该策略解决了多图像、多任务、多区域信息直接汇总导致的上下文压力和信息噪声问题，使报告生成过程更稳定、更可解释。

第三，构建了完整工程闭环。系统通过服务化架构、gRPC 协议、异步 Worker、回调接口、数据库状态机、对象存储、消息队列和 WebSocket 推送，使 Agent 能力从一次性调用变成可运行、可追踪、可恢复的业务流程。这一点体现了本文作为软件工程毕业设计的工作量和系统复杂度。

从毕业设计工作量角度看，本文完成的不只是若干独立功能点，而是一条跨前端、

后端、模型服务、对象存储、消息队列和 Agent 平台的完整链路。系统中任何一个环节缺失，最终用户都无法获得稳定的智能评估体验。因此，本文的主要成果既包括方法设计，也包括对方法的工程落地验证。

7.3 不足之处

本文仍存在不足。首先，五类原子检测能力虽然已经接入系统，但模型本身不是本文重点，尚未围绕真实大规模幕墙数据集进行系统精度评估。其次，Agent 的领域知识主要来自提示词和结构化输入，尚未接入可检索的幕墙规范、专家案例和知识库。再次，报告生成质量仍需要通过更多真实项目和专家评价进行校准。最后，系统作为毕业设计原型，在监报告警、权限精细化、分布式扩展、模型版本管理和报告导出方面仍有继续完善空间。

7.4 未来工作

未来工作可以从四个方向展开。第一，扩充建筑幕墙真实场景数据集，对原子检测能力进行更严格的评估和优化。第二，引入检索增强生成机制，将幕墙工程规范、维护标准和专家案例作为 Agent 的外部知识来源，提高报告专业性。第三，完善系统工程能力，包括分布式 Worker、任务优先级、模型版本切换、监报告警和数据备份。第四，探索与 BIM 平台、无人机巡检平台和建筑运维系统对接，使图像检测结果能够进一步映射到建筑空间模型中。

总体而言，本文验证了 AI Agent 在建筑幕墙韧性评估中的应用价值。Agent 不是替代工程系统，而是通过受控调度和分层信息整合补足传统工程系统在语义判断和报告生成方面的不足。本文系统为建筑幕墙智能运维提供了一条可落地的技术路线，也为 AI Agent 在垂直工程软件中的应用提供了参考。

参考文献

- [1] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[J/OL]. arXiv preprint arXiv:2210.03629, 2023 [2026-05-09]. <https://arxiv.org/abs/2210.03629>.
- [2] SCHICK T, DWIVEDI-YU J, DESSÌ R, et al. Toolformer: Language Models Can Teach Themselves to Use Tools[J/OL]. arXiv preprint arXiv:2302.04761, 2023 [2026-05-09]. <https://arxiv.org/abs/2302.04761>.
- [3] WU Q, BANSAL G, ZHANG J, et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework[J/OL]. arXiv preprint arXiv:2308.08155, 2023 [2026-05-09]. <https://arxiv.org/abs/2308.08155>.
- [4] FREIMUTH H, KÖNIG M. Planning and Executing Construction Inspections with Unmanned Aerial Vehicles[J/OL]. Automation in Construction, 2018, 96: 540-553. DOI: 10.1016/j.autcon.2018.10.016.
- [5] LI Q, PENG X, ZHONG X, et al. Quantitative Identification of Debonding Defects in Building Façades Based on UAV-Thermography Using a Two-Stage Network Integrating Dual Attention Mechanism[J/OL]. Infrared Physics & Technology, 2024, 138: 105241. DOI: 10.1016/j.infrared.2024.105241.
- [6] ZHANG C, WANG F, ZOU Y, et al. Automated UAV Image-to-BIM Registration for Building Façade Inspection Using Improved Generalised Hough Transform[J/OL]. Automation in Construction, 2023, 153: 104957. DOI: 10.1016/j.autcon.2023.104957.
- [7] LECUN Y, BENGIO Y, HINTON G. Deep Learning[J/OL]. Nature, 2015, 521(7553): 436-444. DOI: 10.1038/nature14539.
- [8] HE K, ZHANG X, REN S, et al. Deep Residual Learning for Image Recognition[C/OL] //Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 770-778. DOI: 10.1109/CVPR.2016.90.

-
- [9] YE X W, JIN T, CHEN P Y. Structural Crack Detection Using Deep Learning-Based Fully Convolutional Networks[J]. Advances in Structural Engineering, 2019.
- [10] MEI H, YANG X, WANG Y, et al. Don't Hit Me! Glass Detection in Real-World Scenes [C/OL]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2020: 3684-3693. DOI: 10.1109/CVPR42600.2020.00374.
- [11] WANG L, MA C, FENG X, et al. The Rise and Potential of Large Language Model Based Agents: A Survey[J/OL]. arXiv preprint arXiv:2309.07864, 2023 [2026-05-09]. <https://arxiv.org/abs/2309.07864>.
- [12] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J/OL]. arXiv preprint arXiv:2005.11401, 2020 [2026-05-09]. <https://arxiv.org/abs/2005.11401>.

装
订
线

附录

A. 系统模块与接口补充说明

本附录补充列出系统主要服务模块和检测任务输出内容，便于读者从工程角度理解系统组成。

A.1 服务模块

表 A.1 系统服务模块说明

模块	职责
Core Web	提供项目流程页面、图像资产管理、检测进度可视化、复核与报告展示。
Core API	提供 HTTP 和 WebSocket 入口，负责鉴权、请求解析、响应包装和消息广播。
Core BIZ	承载核心业务逻辑、数据库事务、对象存储、任务 Worker 和事件发布。
Activity Classification	调用分类 Agent，输出需要执行的检测任务代码列表。
Activity Reasoning	调用五类 Python 原子检测能力，上传检测产物并回调检测结果。
Activity Summary	调用总结 Agent，完成单图总结和项目级报告生成。
ICW Common	存放共享协议、枚举、gRPC 工具、日志工具和通用错误结构。

A.2 原子检测任务

表 A.2 原子检测任务与输出结果

任务代码	检测对象	主要输出
corrosion	金属锈蚀	是否存在锈蚀、区域数量、置信度、锈蚀像素、锈蚀占比和区域列表。
crack	石材裂缝	是否存在裂缝、裂缝数量、裂缝像素、裂缝占比和区域列表。
stain	石材污渍	是否存在污渍、污渍区域统计、污渍占比和区域列表。
flatness	玻璃平整度	平整、不平整或非玻璃结论，不平整区域数量和区域指标。
spalling	玻璃爆裂	是否存在爆裂和分类置信度。

谢辞

感谢指导老师在课题选定、系统设计和论文撰写过程中给予的指导与帮助。老师对研究问题、工程实现和论文表达提出的建议，使我能够不断收束课题主线，明确“Agent 自主调度与报告生成”这一核心方向。

感谢同学和朋友在系统开发、界面体验和部署测试过程中给予的建议与反馈。毕业设计涉及前端、后端、模型服务、消息队列、对象存储和部署运维等多个环节，许多问题都是在反复讨论和测试中逐步定位并解决的。

感谢同济大学和计算机科学与技术学院提供的学习平台，使我能够在软件工程专业训练基础上完成一个较完整的工程系统。通过本次毕业设计，我对需求分析、系统架构、服务通信、异步任务和智能体应用有了更系统的理解。

最后，衷心感谢家人在求学期间给予的理解、支持与鼓励。